

Task15 Report

Tomas Cornejo 105320237

In this spike we had to demonstrate a predictive targeting simulation, using a stationary Agent shooting at a patrolling agent using 4 different weapons. This was used using the quadratic formula for predictive movement.

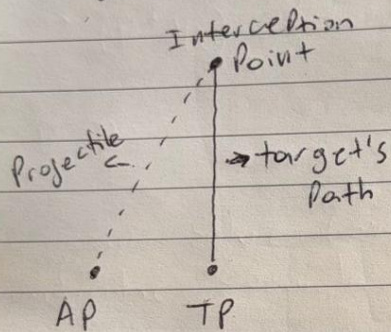
I will now attach pictures on how I did the math for the targeting prediction and overall strategy

AP = Attacker position ~~vs velocity~~

TP = target position

Tv = target velocity

Ps = Projectile speed



t = seconds in time

$$\text{Target_future} = TP + tv * t$$

$$\text{Projectile_Path} = \text{Target_future} - AP$$

Projectile covers that in speed 'Ps':

$$|\text{Target_future} - AP| = Ps * t$$

Turns into:

$$|TP + Tv * t - AP| = Ps * t$$

~~Q2~~

D = the distance vector from T_p to A_p

~~$D = T_p - A_p$~~

$$D = T_p - A_p$$

Previous target-future formula

turns into =

$$|T_p - A_p| + T_v * t = S_p * t$$

which can be replaced with D

$$|D| + T_v * t = S_p * t$$



using $at^2 + bt + c = 0$

terms:

$$a = v_t * v_t - s^2$$

$$b = 2 * (D * v_t)$$

$$c = D * D$$

we can solve for t on
quadratic formula

then, if =

*discriminant ($b^2 - 4ac$) is negative:

- Target too fast (shoot current pos)

*Two positive roots:

- Use smaller one

*both negative:

- Target moving too fast (shoot curr

$a=0$: both moving at same speed

("Shoot current pos" is cut off the picture on the bottom scenarios)

With $t =$

interception point = target pos + target vel $* t$

With this completed I could start coding, I created Projectile.py, Weapon.py, and modified Agent.py

Projectile handles the own projectile movement, lifetime (to not create infinite number of projectiles in a single frame), and visual shape. I didn't attach it to Agents, weapons or anything, it just knows that it has to start, move and then eventually get deleted.

Weapon handles fire rate, projectile speed, accuracy (or the spread angle as it was put on canvas), and the generation of projectiles.

Agent handles steering, patrolling, prediction math, and actually shooting the weapon.

I decided to separate them like this, that way each file makes sure a specific layer of the simulation works. If something specific goes wrong, I know where to look.

The most challenging part of this assignment was figuring out how to transpose the math into my code. I'm able to do it on paper and in theory but when it came to making it a working thing I hit a few walls. I had overwriting positions, inaccurate movement, my def calculate method was failing a lot in attack mode, which made the stationary Agent to never shoot.

I also decided to delete every single Agent mode. The specific task is to make the scenario of a stationary Agent that shoots a patrolling Agent with 4 different options for weapons. Individually changing states, Agents speed, etc. was just causing me more troubleshooting and bugs, so I decided to delete them completely.

It was a fun but challenging task, and I learned a lot on how to apply my math in a different way into my code.